Guide d'Étude Structuré : Architecture et Terminologie `SQL Server`

Niveau 1 : Architecture du `RDBMS` et Écosystème

Comprendre l'environnement fondamental, le langage et les outils de gestion du moteur de base de données.

- **`Database` (Base de données)** : Un macro-récipient logique et physique structuré regroupant des schémas, des entités relationnelles et des métadonnées. Il s'agit du périmètre de sécurité primaire et de l'espace de persistance isolant les transactions pour un domaine d'application spécifique.
- **`SGBD` (Système de Gestion de Base de Données)** : L'infrastructure logicielle globale (comme `SQL Server`) orchestrant le stockage physique, l'intégrité transactionnelle et la gestion de la concurrence. Il virtualise la couche matérielle pour fournir une interface relationnelle logique aux clients.
- **`Instance`** : L'environnement d'exécution logiciel et le processus système isolé contenant les moteurs de stockage et relationnels. Une machine physique peut héberger plusieurs instances fonctionnant de manière totalement indépendante.
- **`SQL` (Structured Query Language)** : Le langage de programmation déclaratif normalisé permettant la communication asynchrone avec le moteur. Le **`SQL` déclaratif** repose sur le principe de spécifier formellement le résultat attendu sans dicter la procédure algorithmique itérative pour l'obtenir.
- **`T-SQL` (Transact-SQL)** : L'implémentation dialectale propriétaire de Microsoft qui étend le standard `SQL` en y incorporant des structures de contrôle procédurales, la gestion des erreurs et des fonctions intrinsèques asymétriques.
- **`SQL Server Management Studio (SSMS)`** : L'environnement de développement intégré (IDE) client fournissant l'interface graphique pour administrer l'architecture, développer le code et analyser les performances d'exécution.

Niveau 2 : Fondations de la Modélisation des Données

Les concepts théoriques et structurels permettant de concevoir et d'organiser l'information.

- **`Modèle EA` (Modèle Entité-Association)** : Le paradigme conceptuel abstrait modélisant les objets métier sous forme d'entités et leurs interactions avant l'implémentation physique. Il sert de plan de conception sémantique.
- **`Schema`** : Un espace de noms logique hiérarchique servant de conteneur de sécurité. Il délimite le regroupement d'entités et sépare la définition de l'objet de son propriétaire, facilitant la gestion des privilèges.
- **`Schéma relationnel`** : La modélisation mathématique définissant la structure intégrale, les domaines et les contraintes de l'ensemble des tables sans contenir de données.
- **Structure d'une `Table`** :
  - **`Table`** : L'instanciation physique et sémantique fondamentale d'une relation, composée d'un schéma d'attributs verticaux et d'un ensemble de tuples.
  - **`Colonne` / `Attribut`** : L'axe vertical définissant un trait spécifique de l'entité, son type de données et ses contraintes pour l'ensemble des tuples.

    - `Ligne` / `Enregistrement` : L'occurrence unique (tuple) d'une entité représentant l'information complète pour un sujet précis.
    - `Champ` : L'intersection atomique minimale d'une colonne et d'une ligne, contenant une valeur scalaire précise ou un marqueur `NULL`.
- `Cardinalité` **et Relations** : Règle quantitative décrivant la nature mathématique de la relation entre deux entités, essentielle pour l'optimiseur de requêtes.
    - `One-to-One` : Lien exclusif imposant qu'une ligne d'une table corresponde univoquement à une seule ligne d'une autre, souvent utilisé pour isoler des données sensibles.
    - `One-to-Many` : La relation asymétrique fondamentale où une ligne parente gouverne une multiplicité de lignes enfants.
    - `Many-to-Many` : Association complexe nécessitant la création d'une table de jonction intermédiaire dans le modèle physique.
- `Normalization` **vs** `Denormalization` : La `Normalization` est la restructuration visant à éliminer toute redondance pour garantir l'intégrité lors des mises à jour. À l'inverse, la `Denormalization` est la réintroduction volontaire de redondances pour optimiser drastiquement les performances de lecture.

Niveau 3 : Types de Données Primitifs

Les formats définissant l'empreinte mémoire et la sémantique de chaque information stockée.

- **Numériques Entiers** :
    - `BIT` : Entier booléen restreint à 0, 1 ou `NULL`.
    - `TINYINT` : Un octet, entiers strictement positifs (0 à 255).
    - `SMALLINT` : Deux octets, pour des domaines de valeurs restreints.
    - `BIGINT` : Huit octets, pour mémoriser des valeurs numériques gigantesques et prévenir les dépassements de capacité.
- **Numériques Exacts et Approximatifs** :
    - `DECIMAL` / `NUMERIC` : Types exacts requérant une déclaration de précision et d'échelle, garantissant l'absence de perte par arrondi.
    - `FLOAT` / `REAL` : Types approximatifs déléguant les calculs à l'unité à virgule flottante pour optimiser la vitesse au détriment de l'exactitude stricte.
    - `money` : Type asymétrique spécifique à `SQL Server` pour l'arithmétique financière.
- **Chaînes de Caractères** :
    - `CHAR` : Chaîne de longueur statique, remplissant les vides avec des espaces.
    - `VARCHAR` : Chaîne à longueur dynamique minimisant l'empreinte de stockage physique.
    - `NCHAR` / `NVARCHAR` : Équivalents utilisant l'encodage `Unicode` (doublant l'espace consommé) pour supporter tous les alphabets internationaux.
    - `TEXT` / `NTEXT` : Anciens types dépréciés pour les très longs textes, remplacés techniquement par `VARCHAR(MAX)` et `NVARCHAR(MAX)`.
- **Temporels** :
    - `TIME` : Sérialise isolément l'heure sans contexte calendaire.
    - `DATETIME` / `DATETIME2` : Modélisent simultanément la date et l'heure. `DATETIME2` offre une plage temporelle et une précision fractionnelle accrues.
    - `SMALLDATETIME` : Format allégé arrondissant les secondes à la minute la plus proche.
- **Spécialisés** :
    - `VARBINARY` : Sérialisation brute de données binaires (`BLOB` comme des images ou fichiers).

    - `XML data type` : Permet l'hébergement de documents balisés hiérarchiques, agissant comme un mini-moteur orienté document.
    - `UNIQUEIDENTIFIER` : Type de 16 octets générant des codes `GUID` cryptographiques pour garantir l'unicité décentralisée.

Niveau 4 : Data Definition Language (`DDL`) et Contraintes

Les instructions permettant de forger, d'altérer et de sécuriser la structure du schéma.

- **`DDL` (Data Definition Language)** : Sous-ensemble de commandes (`CREATE`, `ALTER`, `DROP`) interagissant avec le dictionnaire de données pour dicter l'architecture physique, sans manipuler le contenu des lignes.
    - **`CREATE TABLE`** : Construit une nouvelle entité et alloue son espace métadonnées.
    - **`ALTER TABLE`** : Modifie structurellement une entité existante (ajout de colonnes, etc.).
    - **`DROP TABLE`** : Détruit définitivement l'entité et dé-alloue ses pages de données.
    - **`TRUNCATE TABLE`** : Vide intégralement et instantanément une table par dé-allocation de blocs de mémoire, sans journaliser la suppression ligne par ligne.
- **`CONSTRAINT`** : Directive d'intégrité préservant la validité sémantique de la base :
    - **`PRIMARY KEY`** : Identifiant unique absolu pour chaque ligne, interdisant le `NULL`.
    - **`UNIQUE`** : Garantit qu'une valeur n'est jamais dupliquée au sein d'une colonne (accepte `NULL`).
    - **`NOT NULL`** : Interdit formellement l'absence de valeur ou l'insertion du marqueur vide.
    - **`DEFAULT`** : Fournit une valeur scalaire prédéfinie de remplacement lors d'un oubli d'insertion.
    - **`CHECK`** : Impose une expression logique booléenne stricte validant le domaine de données de chaque insertion.
- **`IDENTITY` / `Auto-incrémentation`** : Mécanisme interne générant asynchroniquement une séquence numérique unique pour assurer l'identité des nouveaux enregistrements sans intervention manuelle.

Niveau 5 : Data Manipulation (`DML`) et Extraction (`DRL`)

Les commandes opérationnelles permettant l'insertion, la lecture, la modification et la suppression de l'information.

- **`DML` (Data Manipulation Language)** : Instructions interagissant avec les données contenues dans les tables. Le **`DRL` (Data Retrieval Language)** en est le sous-ensemble dédié exclusivement à l'extraction de données sans altération de l'état persistant (`SELECT`).
- **`SELECT` & `FROM`** : L'instruction `SELECT` est le processeur d'interrogation traduisant la demande en arbre d'exécution physique. La clause `FROM` identifie formellement la ou les tables sources à lire.
- **`INSERT INTO` & `VALUES`** : L'instruction `INSERT INTO` instancie physiquement de nouveaux tuples, tandis que la clause `VALUES` permet d'énumérer de manière explicite les données brutes à y injecter.
- **`DELETE FROM`** : Exécute la suppression irréversible de tuples existants de manière transactionnelle.
- **`OUTPUT`, `INSERTED` & `DELETED`** : La clause `OUTPUT` permet de récupérer immédiatement l'état des données venant d'être mutées. Elle s'appuie sur le concept de `INSERTED` et `DELETED`, qui sont des tables virtuelles éphémères exposant respectivement les nouvelles valeurs entrantes et les anciennes valeurs écrasées/supprimées.
- **`DUAL`** : Bien que moins pertinent sous `SQL Server` (qui s'en affranchit), il s'agit conceptuellement d'une entité factice employée pour forcer la validité syntaxique d'un `SELECT` calculant de simples valeurs scalaires.

Niveau 6 : Filtrage, Logique et Fonctions Intégrées

Les outils algorithmiques pour restreindre les données, opérer des calculs et transformer les flux.

- **Filtrage Logique** :
  - `WHERE` : Clause fondamentale implémentant un prédicat pour filtrer les lignes selon des conditions.
  - `AND` / `OR` : Opérateurs binaires modifiant le comportement du filtrage (`AND` est restrictif, `OR` est expansif). L'ordre de ces opérations est strictement régi par la `Précédence des opérateurs`.
  - `IN` / `BETWEEN` : L'opérateur `IN` vérifie l'appartenance à un ensemble de valeurs, tandis que `BETWEEN` vérifie l'inclusion dans un intervalle fermé.
  - `LIKE` & `Wildcard` : Permettent l'évaluation textuelle via des modèles asymétriques à l'aide de métacaractères (`Wildcards` comme `%`).
  - `IS (NOT) NULL` : Le seul opérateur capable d'évaluer de manière déterministe l'absence absolue de donnée.
- **Formatage du Résultat** :
  - `ORDER BY` (`ASC`, `DESC`) : Restructure physiquement le flux de données final par ordre croissant (`ASC`) ou décroissant (`DESC`).
  - `DISTINCT` : Filtre le flux de sortie pour éliminer les doublons, garantissant l'unicité des lignes.
  - `Alias` : Surnom temporaire optimisant la résolution lexicale des colonnes ou tables (via `AS`).
- **Évaluations Conditionnelles** :
  - `CASE...WHEN...THEN...ELSE...END` : Expression vectorielle retournant une valeur selon la première condition logique remplie.
  - `COALESCE` : Retourne la première valeur non-`NULL` d'une liste évaluée séquentiellement.
  - `NULLIF` : Retourne `NULL` préventivement si deux valeurs sont mathématiquement égales.
- **Fonctions Scalaires et Transformations** :
  - **Type** : `CAST` et `CONVERT` forcent la traduction explicite d'un type de donnée (par exemple de texte à numérique).
  - **Mathématiques** : `ABS` (valeur absolue positive), `Modulo` (reste de la division euclidienne).
  - **Texte** : `LEN` (longueur absolue), `LOWER` / `UPPER` (normalisation minuscule/majuscule), `LTRIM` / `RTRIM` (nettoyage des espacements), `SUBSTRING` (extraction positionnelle), `CHARINDEX` (position d'une sous-chaîne), `REPLACE` (substitution), `REPLICATE` (concaténation répétitive), `Concaténation` (fusion asymétrique de chaînes).
  - **Temporel** : `GETDATE()` (horloge système actuelle), `DATEADD` / `DATEDIFF` (manipulation et écart chronologique), `DATEPART` (extraction granulaire d'une dimension, ex: mois).

```sql
SELECT e.Name AS EmployeeName FROM Employees AS e;
SELECT ABS(-42) AS AbsoluteValue;
```

Niveau 7 : Agrégation et Groupement

La ségrégation et la consolidation mathématique des flux de données.

- `GROUP BY` : Ségrègue le flux en sous-ensembles selon l'équivalence des attributs dimensionnels.
- `HAVING` : Filtre les sous-ensembles générés *après* l'application de l'agrégation, contrairement au `WHERE`.
- **Fonctions Mathématiques Associées** :
  - `COUNT` : Calcule la cardinalité totale (nombre de lignes).
  - `SUM` / `AVG` : Effectuent la consolidation arithmétique globale (somme ou moyenne) en ignorant les `NULL`.
  - `MIN` / `MAX` : Identifient l'extrémité scalaire la plus faible ou la plus élevée d'une distribution.

Niveau 8 : Mécanique Relationnelle et Jointures

Les concepts permettant de naviguer dans l'arborescence des tables et de fusionner l'information.

- `Intégrité référentielle` **et** `FOREIGN KEY` : Paradigme garantissant qu'une clé étrangère pointe de manière absolue vers un tuple existant. Géré automatiquement par les règles de propagation `ON DELETE` / `ON UPDATE` (`CASCADE`, `SET NULL`, `SET DEFAULT`, `NO ACTION`).
- `Jointures horizontales / verticales` : La jointure horizontale ajoute des colonnes (ex: `INNER JOIN`), tandis que la verticale ajoute des lignes (ex: `UNION`).
- **Types de Jointures Horizontales** :
  - `INNER JOIN`, `EQUI-JOIN` & `NON EQUI-JOIN` : L'`INNER JOIN` produit une intersection stricte. L'`EQUI-JOIN` se base sur une stricte égalité, tandis que le `NON EQUI-JOIN` s'appuie sur des inéquations.
  - `LEFT OUTER JOIN` / `RIGHT OUTER JOIN` : Préservent l'intégralité asymétrique de l'entité située à gauche ou à droite, complétant les données manquantes par des `NULL`.
  - `FULL OUTER JOIN` : Assure la préservation bilatérale totale des deux ensembles.
  - `CROSS JOIN` / `Produit cartésien` : Fusion exhaustive de chaque ligne d'une table avec chaque ligne de la seconde, causant une explosion exponentielle de la cardinalité.
  - `SELF-JOIN` : Corrélation d'une table avec elle-même pour résoudre des hiérarchies récursives.

Niveau 9 : Requêtes Avancées et Opérations Ensemblistes

Les manipulations complexes, la récursivité et l'algèbre d'ensembles poussée.

- `Sous-requêtes` : Requêtes imbriquées agissant comme opérateurs de recherche. Elles peuvent être scalaires (une valeur), multi-valeurs, tabulaires, ou **Corrélées** (évaluées itérativement via une dépendance lexicale à la requête parente).
- **Quantificateurs Logiques** :
  - `EXISTS` : Vérifie simplement si une sous-requête retourne au moins un tuple (Semi-Join).
  - `ANY` / `ALL` : Comparateurs vérifiant si une condition est vraie pour "au moins un" ou pour "la totalité" des éléments d'un ensemble de résultats.
- **Opérateurs Ensemblistes (Jointures Verticales)** :
  - `UNION [ALL]` : Concatène verticalement deux flux de même arité. `ALL` inhibe la lourde étape de déduplication.
  - `INTERSECT` : Ne conserve que les lignes strictement présentes dans les deux flux.
  - `EXCEPT` / `MINUS` : Opération de soustraction retirant du premier flux tout ce qui apparaît dans le second.

- `WITH` / `CTE (Common Table Expression)` : Expression nommée générant une table virtuelle temporaire très lisible, extrêmement puissante pour formuler des requêtes récursives.
- `MERGE` / `MERGE INTO` **(Fusion de données)** : Super-commande orchestrant simultanément l'insertion, la modification et la suppression de lignes entre deux tables selon une condition de corrélation asymétrique.
- `CUBE / ROLLUP` : Extensions de `GROUP BY` générant automatiquement des super-agrégations hiérarchiques et des sous-totaux dynamiques dans le flux.

Niveau 10 : Transactions, Concurrence et État

Garantir la fiabilité absolue des mutations et prévenir le chaos lors des accès simultanés.

- `ACID` **(Atomique, Cohérente, Isolée, Durable)** : Paradigme strict assurant que les mutations s'exécutent entièrement et résistent aux pannes.
- **Gestion Transactionnelle** :
  - `BEGIN TRANSACTION` : Initie la limite logique d'un groupe d'opérations.
  - `COMMIT` : Applique de manière permanente et irréversible les modifications validées.
  - `ROLLBACK` : Annule atomiquement toutes les mutations depuis le début en cas de défaillance.
  - `AUTO-COMMIT` : Mode par défaut validant immédiatement chaque instruction réussie de manière isolée.
- `Verrous (Locks)` & `DEADLOCK` : Les verrous sont des barrières de sécurité virtuelles garantissant l'isolation des données. Un `DEADLOCK` est un état critique où deux transactions s'attendent mutuellement indéfiniment, forçant le moniteur système à sacrifier l'une d'elles.

Niveau 11 : Programmabilité et Automatisation

Encapsuler la logique métier complexe au cœur du système de base de données.

- `View` : Entité virtuelle persistée agissant comme une fenêtre d'abstraction de requêtage, masquant la complexité du modèle physique.
- `User-Defined Function (UDF)` : Routine mathématique ou tabulaire encapsulée par le développeur, incapable de modifier l'état persistant de la base de données (lecture seule).
- `SQL Server Procedure` : Bloc précompilé complet gérant la complexité transactionnelle et algorithmique. Contrairement à une fonction, il peut muter l'état et utiliser `TRY...CATCH`.
- `Trigger` : Code T-SQL événementiel asynchrone se déclenchant impérativement et automatiquement lors d'une mutation de données, s'appuyant souvent sur l'état des tables virtuelles `INSERTED` et `DELETED`.

Niveau 12 : Architecture du Moteur, Stockage et Administration

L'anatomie interne, l'optimisation physique et les stratégies de maintenance critique.

- **Architecture de Compilation** :
  - `Parser` : Traducteur initial validant la sémantique et la syntaxe lexicale de la requête.
  - `Query Optimizer` : Le "cerveau" heuristique générant l'`Execution Plan` (la carte des instructions binaires) le plus rapide en fonction des statistiques mathématiques.
  - `Query Processor` & `Query Executor` : L'infrastructure logicielle appliquant l'arbre d'exécution pour orchestrer les jonctions et réclamer les données au stockage.
- **Gestionnaires et Background** :

  - `Storage Engine` : Sous-système contrôlant les verrous, les accès disques et les journaux de transaction.
  - `Database Manager` & `SQL Manager` : Superviseurs allouant la mémoire, gérant les fichiers et gérant la mémoire cache des plans d'exécution.
  - `SQL Server Agent` : L'orchestrateur système chargé d'exécuter asynchrone les tâches planifiées et la télémétrie.
- **Physique du Stockage** :
  - `Page (8 KB)` & `Extent` : La `Page` est la brique minimale d'entrée/sortie sur disque (8192 octets). Un `Extent` regroupe 8 pages contiguës pour prévenir la fragmentation matérielle.
  - **Fichiers (.mdf, .ndf, .ldf)** : Le fichier `.mdf` est le dictionnaire de persistance principal. Le fichier `.ndf` sert d'expansion de stockage secondaire. Le fichier `.ldf` est le journal de bord transactionnel sérialisant chronologiquement toutes les mutations (Write-Ahead Logging).
- **Optimisation, Versionnage et Métadonnées** :
  - `Index` **(**`Clustered` **/** `Non-Clustered Index`**)** : Arbre algorithmique (`B-Tree`). Le type `Clustered` trie l'ordre physique réel du stockage, tandis que le type `Non-Clustered` maintient une structure pointant vers l'emplacement des données.
  - `ROWVERSION` : Marqueur binaire opaque s'actualisant seul pour prévenir les conflits de concurrence.
  - `Collation` : Règles dictant l'encodage lexicographique (sensibilité à la casse, tris internationaux).
  - `SET` : Instruction polyvalente modifiant l'état d'un attribut ou les options de session.
- **Administration de Haute Disponibilité** :
  - `Backup` **&** `Recovery Model` : Les sauvegardes et les modèles (ex: Simple, Full) régissent la résilience et le comportement d'archivage des journaux.
  - `Log Shipping` / `Mirroring` / `Replication` : Architectures de redondance asynchrone ou synchrone répliquant le flux de transactions ou des entités spécifiques vers d'autres serveurs du réseau.